

CAPSTONE PROJECT: Learning Management & Prerequisite Tracking System (LMPTS)

Problem Statement

Context

Educational institutions and corporate training programs face a critical challenge: managing complex course prerequisite structures at scale. Currently, most systems handle prerequisites manually or use rigid, inflexible approaches that:

- Fail to prevent students from enrolling in courses before completing prerequisites
- Cannot detect circular dependency chains (A requires B, B requires C, C requires A)
- Lack intelligent algorithms to recommend optimal learning sequences
- Provide limited visibility into learner progress across prerequisite chains
- Are tightly coupled to specific database technologies, making them difficult to extend

The Challenge

Design and implement a **Learning Management & Prerequisite Tracking System (LMPTS)** that:

1. **Manages a catalog of courses** with metadata (code, name, difficulty, duration, instructor)
2. **Enforces prerequisite constraints** ensuring students only enroll when all dependencies are met
3. **Detects circular dependencies** and prevents them from being added to the system
4. **Tracks learner progress** across courses and calculates completion metrics
5. **Recommends learning paths** using intelligent algorithms
6. **Generates analytics** on course completion rates, learner performance, and system metrics
7. **Provides a user interface** for administrators, instructors, and learners

System Actors

- **Administrator** - Creates/manages courses, defines prerequisites, views system metrics
 - **Instructor** - Views student progress, manages enrollments, tracks completion
 - **Learner** - Views available courses, enrolls in courses, tracks personal progress
 - **Analyst** - Generates reports on completion rates and learning bottlenecks
-

Learning Objectives

After completing this capstone, you will demonstrate:

OOP Mastery

- ☐ **Inheritance** - Design class hierarchies with abstract base classes
- ☐ **Encapsulation** - Implement private attributes with controlled access via properties
- ☐ **Polymorphism** - Create multiple implementations of interfaces
- ☐ **Abstraction** - Hide implementation complexity behind interfaces
- ☐ **Composition** - Combine simple objects into complex structures
- ☐ **Proper modeling** - Design domain objects that reflect real-world entities

Data Structures & Algorithms

- ☐ **Directed Graph** - Implement a prerequisite network with adjacency representation
- ☐ **Queue (FIFO)** - Use BFS algorithm for shortest-path computation
- ☐ **Set operations** - Implement efficient membership testing and set logic
- ☐ **Dictionary/Hash Maps** - Optimize lookups and caching
- ☐ **Tree traversal** - Implement curriculum hierarchies with recursive traversal
- ☐ **Algorithm analysis** - Understand time/space complexity of your implementations

Design Patterns

- ☐ **Repository Pattern** - Abstract database operations from business logic
- ☐ **Factory Pattern** - Centralize object creation with validation
- ☐ **Strategy Pattern** - Implement multiple algorithms with runtime selection
- ☐ **Observer Pattern** - Decouple event producers from consumers
- ☐ **Singleton Pattern** - Manage shared resources safely

Software Engineering

- ☐ **Layered Architecture** - Separate concerns (GUI, services, data access)
 - ☐ **Error Handling** - Create domain-specific exceptions with meaningful messages
 - ☐ **Data Validation** - Enforce business rules at object creation
 - ☐ **Automated Testing** - Write unit, integration, and edge case tests
 - ☐ **Database Design** - Create normalized schemas with relationships
 - ☐ **Code Organization** - Structure code for readability and maintainability
-

Requirements

Functional Requirements

FR1: Course Management

- ☐ Create courses with code, name, description, difficulty level, duration
- ☐ Update course metadata
- ☐ Delete courses (cascade to related records)
- ☐ Retrieve course details and catalog
- ☐ Track course status (draft, published, archived)

FR2: Prerequisite Management

- ☐ Add prerequisite relationships between courses
- ☐ Remove prerequisite relationships
- ☐ View all prerequisites for a course (direct and transitive)
- ☐ Detect and prevent circular dependencies
- ☐ Calculate course levels in the prerequisite hierarchy

FR3: Learner Management

- ☐ Register learner profiles
- ☐ Track learner enrollment history
- ☐ Calculate completion rates and progress metrics
- ☐ View completed and in-progress courses

FR4: Enrollment & Validation

- ☐ Enroll learners in courses
- ☐ Validate that all prerequisites are completed before enrollment
- ☐ Prevent duplicate enrollments
- ☐ Mark courses as completed with optional scores
- ☐ Suggest available courses based on completed prerequisites

FR5: Learning Path Computation

- ☐ Find shortest learning sequence from one course to target course
- ☐ Validate proposed learning paths against prerequisite constraints
- ☐ Provide multiple path suggestions (different strategies)
- ☐ Calculate path statistics (total hours, difficulty progression)

FR6: Analytics & Reporting

- ☐ Generate course completion statistics
- ☐ Calculate system-wide metrics (total courses, learners, completion rate)
- ☐ Analyze prerequisite structure depth and complexity
- ☐ Identify bottleneck courses (high prerequisites, low completion)

FR7: User Interface

- ☐ Admin panel for course/prerequisite management

- ☐ Learner portal for self-service enrollment and progress tracking
- ☐ Analytics dashboard with key metrics
- ☐ Real-time validation feedback (e.g., prerequisite violations)

Non-Functional Requirements

NFR1: Reliability

- ☐ System must prevent invalid prerequisite structures (no cycles)
- ☐ All data mutations must maintain referential integrity
- ☐ Transactions must be atomic (all-or-nothing)

NFR2: Performance

- ☐ Course lookup: $O(1)$ expected time
- ☐ Prerequisite validation: $O(n)$ where n = number of prerequisites
- ☐ Path finding: $O(V + E)$ where V = courses, E = prerequisites
- ☐ Support at least 10,000 courses and 100,000 learners

NFR3: Maintainability

- ☐ Database technology should be abstracted behind repository layer
- ☐ Business logic should not depend on UI framework
- ☐ Algorithm selection should be pluggable (different path-finding strategies)
- ☐ Code should be 80%+ test coverage

NFR4: Extensibility

- ☐ New pathfinding algorithms should be addable without modifying existing code
 - ☐ New course attributes should be simple to add
 - ☐ Database should be swappable (SQLite → PostgreSQL)
 - ☐ GUI should be replaceable (Tkinter → Flask → React)
-

Technical Stack Guidance

Database

- **Primary:** SQLite (included in Python)
- **Abstraction:** Repository pattern (don't hardcode SQL in services)
- **Schema:** Normalized with proper relationships (1:N, M:N)

GUI

- **Framework:** Tkinter (built-in, no installation)
- **Architecture:** MVC - separate models, views, and controllers

- **Binding:** Implement observer pattern for reactive updates

Testing

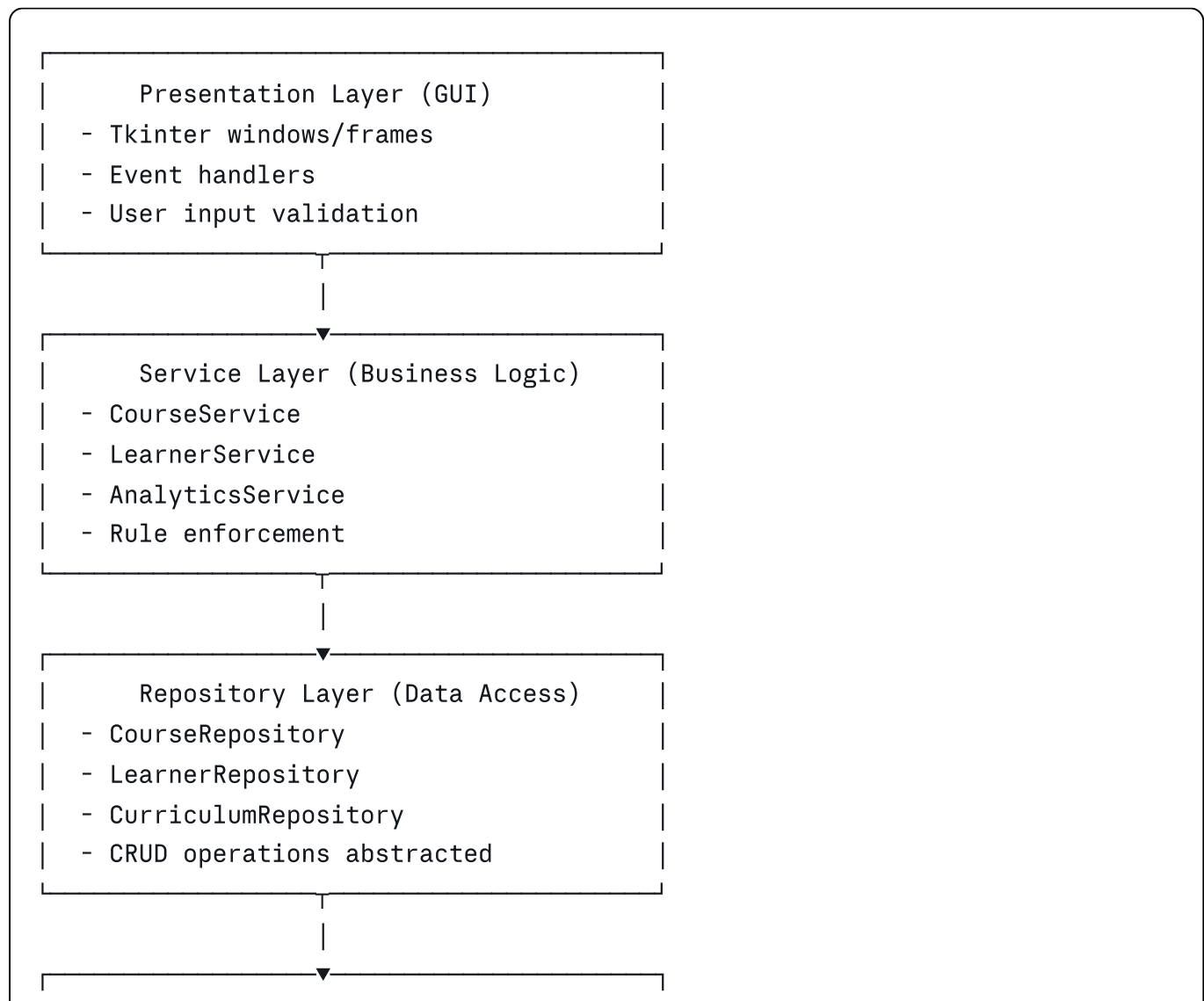
- **Framework:** pytest (pip install pytest)
- **Coverage:** pytest-cov (pip install pytest-cov)
- **Strategy:** Unit tests (60%), integration tests (30%), edge cases (10%)

Language Features

- **Type hints:** Use for documentation and IDE support
 - **Dataclasses:** For lightweight value objects
 - **Enums:** For categorical data (DifficultyLevel, Status)
 - **ABC (Abstract Base Classes):** For interfaces and base classes
 - **Properties:** For encapsulation with controlled access
-

Architecture Guidance

Recommended Layered Architecture



	Database Layer	
	- SQLite connection management	
	- SQL execution	

Domain Layer (Independent)

	Models (Course, Learner, Enrollment)	
	Value Objects (Status enums)	
	Exceptions	
	Algorithms (path finding, validation)	

Module Organization

```

lmpts_project/
├── core/                                # Domain models (independent of framework)
│   ├── models.py                       # OOP classes with inheritance
│   ├── exceptions.py                   # Custom exception hierarchy
│   └── algorithms.py                   # Standalone algorithms
│
├── data/                                # Data access layer (Repository pattern)
│   ├── database.py                     # SQLite management (Singleton)
│   └── repository.py                   # CRUD abstractions
│
├── services/                            # Business logic (orchestration)
│   └── lmpts_service.py                # CourseService, LearnerService, etc.
│
├── gui/                                 # Presentation (Tkinter)
│   └── main.py                         # Application window and views
│
├── tests/                              # Automated tests
│   ├── test_models.py
│   ├── test_data_access.py
│   ├── test_services.py
│   └── test_integration.py
│
├── main.py                             # Application entry point
└── requirements.txt

```

Implementation Tips & Best Practices

1. Start with Models (Test-Driven Approach)

```
python
```

```
# Define what courses look like
class Course:
    def __init__(self, code: str, name: str, ...):
        # Validate in __init__
        if not code:
            raise ValueError("Code cannot be empty")
        self._code = code # Immutable
        self._prerequisites = set() # Mutable via methods only

    @property
    def code(self):
        return self._code # Read-only

    def add_prerequisite(self, code: str):
        # Controlled mutation
        if code == self._code:
            raise ValueError("Self-prerequisite not allowed")
        self._prerequisites.add(code)
```

2. Implement Validation Early

- Validate at object creation (prevent invalid objects from existing)
- Use custom exceptions for domain errors
- Fail fast with clear error messages

```
python

class PrerequisiteNotMetError(Exception):
    """Raised when learner tries to enroll without prerequisites."""
    pass
```

3. Use Abstract Base Classes for Contracts

```
python

from abc import ABC, abstractmethod

class Repository(ABC):
    @abstractmethod
    def create(self, entity):
        pass

    @abstractmethod
    def read(self, id):
        pass
```

4. Encapsulation Pattern

python

```
class Course:
    def __init__(self):
        self._prerequisites = set() # Private

    @property
    def prerequisites(self):
        return self._prerequisites.copy() # Return copy to prevent external mo

    def has_prerequisite(self, code: str) -> bool:
        return code in self._prerequisites # O(1) membership test
```

5. Service Layer Orchestration

python

```
class CourseService:
    def __init__(self, repository, graph):
        self.repository = repository
        self.graph = graph # Algorithm component

    def add_prerequisite(self, prereq: str, dependent: str):
        # 1. Validate both courses exist
        if not self.repository.read(prereq):
            raise CourseNotFoundError(f"Course {prereq} not found")

        # 2. Update algorithm
        try:
            self.graph.add_edge(prereq, dependent)
        except CycleException:
            raise CircularDependencyError(f"Would create cycle")

        # 3. Persist
        self.repository.add_prerequisite(prereq, dependent)
```

6. Testing Strategy

python


```

# Unit test: single component
def test_course_validation_empty_code():
    with pytest.raises(ValueError):
        Course("", "Name", ...)

# Integration test: workflow
def test_enrollment_with_prerequisites():
    # 1. Complete prerequisite
    # 2. Verify available courses updated
    # 3. Enroll in dependent course
    # 4. Verify enrollment succeeded

# Edge case test
def test_empty_prerequisites_set():
    course = Course("CS101", ...)
    assert course.prerequisites == set()

```

7. Observer Pattern for Events

```

python

class ProgressObserver(ABC):
    @abstractmethod
    def on_enrollment_created(self, learner_id, course_code):
        pass

class LearnerService:
    def __init__(self):
        self.observers = []

    def register_observer(self, observer):
        self.observers.append(observer)

    def enroll(self, learner_id, course_code):
        # ... enrollment logic ...
        for observer in self.observers:
            observer.on_enrollment_created(learner_id, course_code)

```

8. Factory Pattern for Object Creation

```

python

```

```
class CourseService:
    def create_course(self, code, name, desc, difficulty, hours):
        # Centralize creation with validation
        course = Course(code, name, desc, difficulty, hours)
        is_valid, msg = course.validate()
        if not is_valid:
            raise EntityValidationError(msg)

        self.repository.create(course)
        return course
```

Grading Rubric

Code Quality (30%)

- ☐ Proper OOP design (inheritance, encapsulation, polymorphism)
- ☐ Clean architecture (layered separation of concerns)
- ☐ Error handling (custom exceptions, validation)
- ☐ Code organization (logical module structure)
- ☐ Documentation (docstrings, comments on complex logic)

Functionality (30%)

- ☐ All requirements implemented and working
- ☐ Prerequisite validation enforced
- ☐ Circular dependency detection works
- ☐ Learning path computation produces valid results
- ☐ Analytics calculations are correct

Testing (20%)

- ☐ Comprehensive test coverage (80%+)
- ☐ Unit tests for individual components
- ☐ Integration tests for workflows
- ☐ Edge cases identified and tested
- ☐ Tests pass consistently

Design Patterns & Data Structures (15%)

- ☐ Appropriate data structures used (why use Set for prerequisites?)
- ☐ Design patterns applied correctly (Repository, Factory, Strategy, Observer)
- ☐ Algorithm complexity understood
- ☐ Extensibility demonstrated (can add new features easily)

User Interface (5%)

- ☐ Functional and intuitive
 - ☐ Proper separation from business logic
 - ☐ Real-time validation feedback
 - ☐ Error messages are helpful
-

Deliverables

Code Submission

```
Impts_project/  
├── core/  
│   ├── models.py  
│   ├── exceptions.py  
│   └── algorithms.py  
├── data/  
│   ├── database.py  
│   └── repository.py  
├── services/  
│   └── lmpts_service.py  
├── gui/  
│   └── main.py  
├── tests/  
│   └── test_suite.py  
├── main.py  
├── requirements.txt  
└── README.md
```

Documentation

- ☐ README with setup and usage instructions
- ☐ Design document explaining architecture and patterns used
- ☐ Test report showing coverage and test results
- ☐ Code comments explaining complex logic

Demonstration

- ☐ Live demo of admin creating courses and prerequisites
 - ☐ Live demo of learner enrolling with prerequisite validation
 - ☐ Run full test suite showing coverage
-

Success Criteria

You have successfully completed this capstone if:

- ✓ **System is fully functional** - All requirements work correctly
 - ✓ **Code is well-designed** - Uses OOP principles and design patterns appropriately
 - ✓ **Tests are comprehensive** - 80%+ coverage with meaningful tests
 - ✓ **Architecture is clean** - Layered design allows easy modification
 - ✓ **Documentation is clear** - Code is understandable and maintainable
 - ✓ **Edge cases handled** - System doesn't crash on invalid input
 - ✓ **Performance is acceptable** - Operations complete in reasonable time
 - ✓ **Extensibility demonstrated** - New features can be added easily
-

Getting Started

Step 1: Design Phase

- ☐ Sketch class hierarchies on paper
- ☐ Identify what data needs to persist
- ☐ Plan module organization
- ☐ List what will be tested

Step 2: Core Implementation

- ☐ Implement domain models first (Course, Learner, etc.)
- ☐ Write tests as you build models
- ☐ Implement repository layer (abstract data access)
- ☐ Implement service layer (business logic)

Step 3: Algorithms

- ☐ Implement prerequisite algorithm (hint: think about cycle detection)
- ☐ Implement path finding (hint: what data structure finds shortest paths?)
- ☐ Test algorithms thoroughly

Step 4: User Interface

- ☐ Create admin panel for course management
- ☐ Create learner portal for self-service
- ☐ Implement validation feedback
- ☐ Add analytics views

Step 5: Testing & Polish

- ☐ Achieve 80%+ test coverage
- ☐ Test integration workflows

- ☐ Handle edge cases
 - ☐ Write clear documentation
-

References & Resources

OOP Design

- "Head First Design Patterns" - Freeman & Freeman
- "Refactoring" - Martin Fowler
- Python ABC documentation

Data Structures & Algorithms

- "Data Structures and Algorithms in Python" - Goodrich et al.
- Python collections module (deque, defaultdict, namedtuple)

Software Engineering

- "Clean Code" - Robert C. Martin
- "The Pragmatic Programmer" - Hunt & Thomas

Python Tools

- pytest documentation
 - SQLite tutorial
 - Tkinter tutorial
-

Questions to Guide Your Implementation

1. How should prerequisites be stored to enable $O(1)$ membership testing?
 2. What algorithm can detect if adding an edge would create a cycle?
 3. How can you ensure only valid objects exist (validation in init)?
 4. How would you replace SQLite with PostgreSQL without changing your services?
 5. How can the GUI update automatically when enrollment happens?
 6. What tests would catch a broken prerequisite check?
 7. How could you support different path-finding strategies (shortest, difficulty-progressive)?
-

Final Note

This capstone is not about writing clever code. It's about designing systems that are:

- **Correct** - Enforce business rules (can't enroll without prerequisites)
- **Clear** - Easy to understand and modify
- **Tested** - Confidence that it works
- **Extensible** - New features are straightforward to add

Focus on design and architecture first. Code is secondary.

Good luck! 🎓